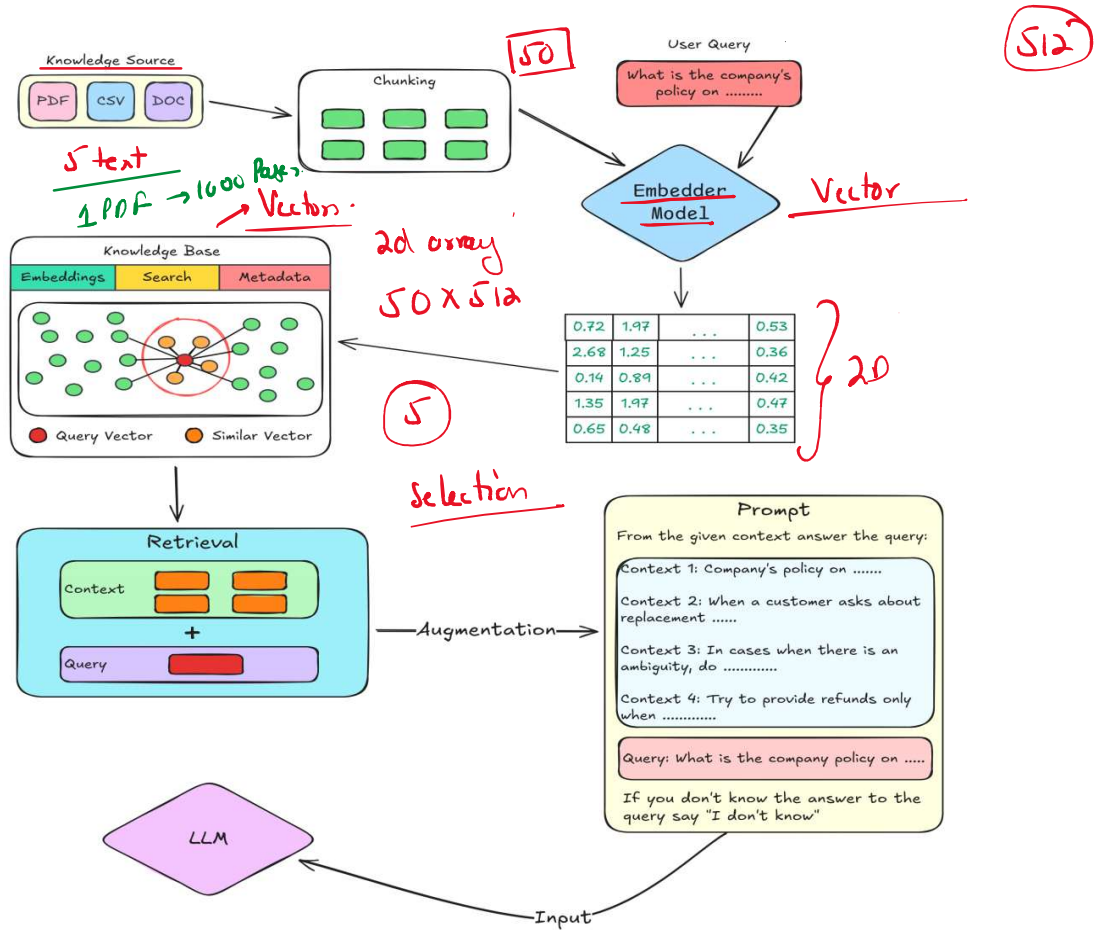


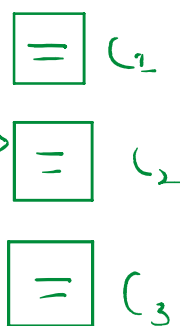
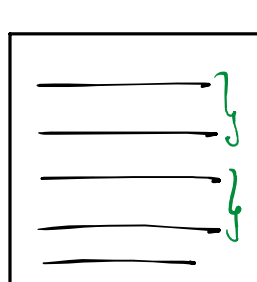
Why need Text Splitters

Each document $\rightarrow$ 10 parts nd $\rightarrow$ Embedding model.



Large Text $\xrightarrow{\text{split}}$ Multiple chunks

Str / Document



I/P $\rightarrow$ Max tokens

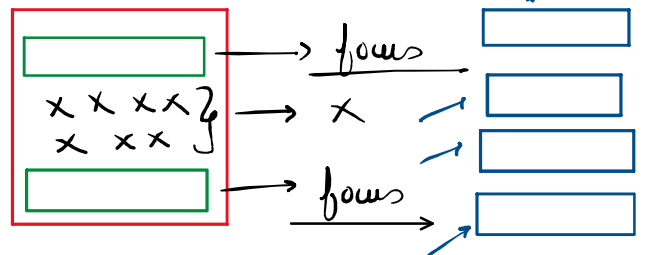
1M words / token

Context window
200k.

Hallucinate

lost in the middle

1L words
100k

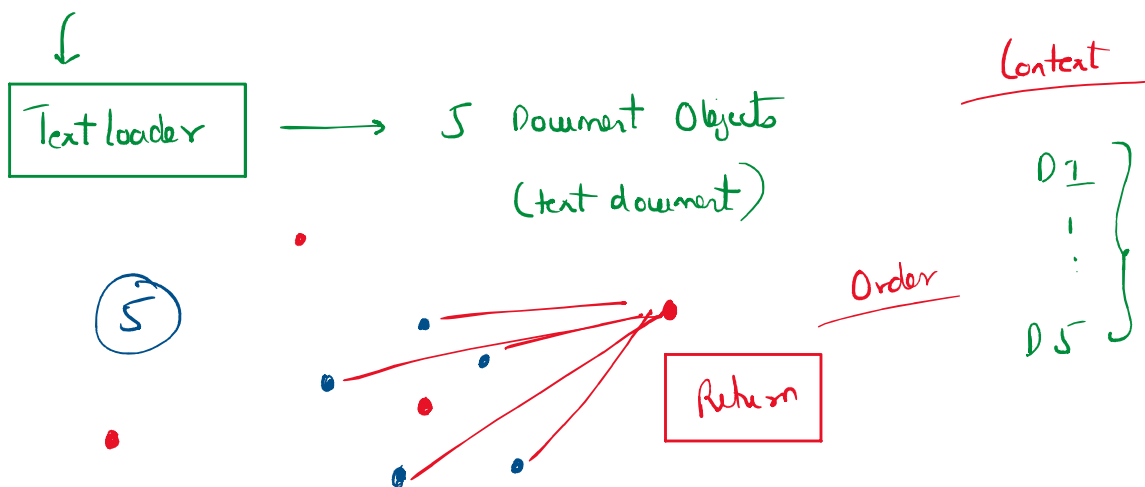


splitting → ① Never breach the context limit
 → ② Avoid the lost in the middle problem.

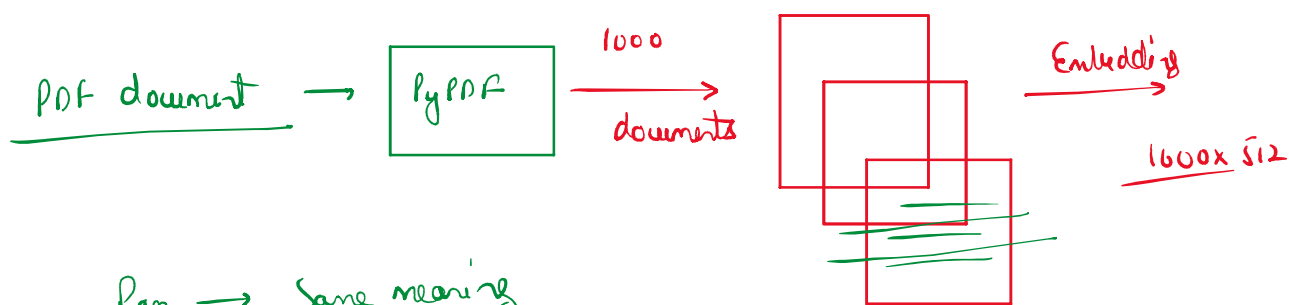
Downstream Processes

1000 lines
 50 → 100 lines
5000 lines
500 ↓ 10x

Knowledge source → 5 text files (knowledge)



Not specific to query.



Page → Same meaning

Semantic meaning chunking.

● Semantic Meaning

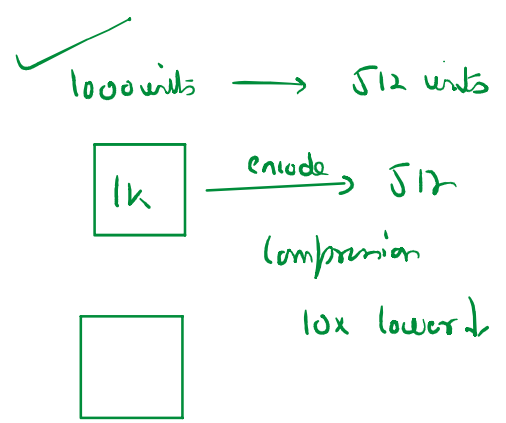
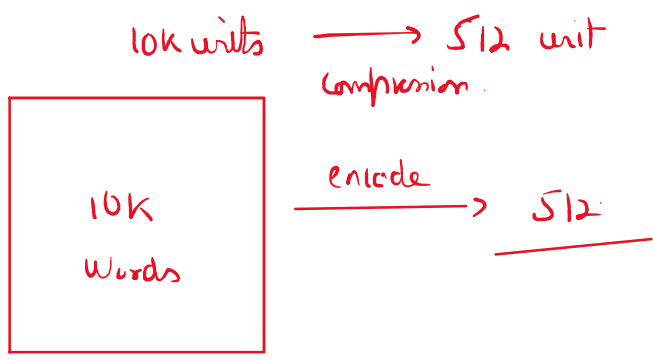
Embedding models → fixed output dimensionality

512

— semantic



Text $\xrightarrow[\text{meaning}]{\text{semantic}}$ encode into 512 numbers.

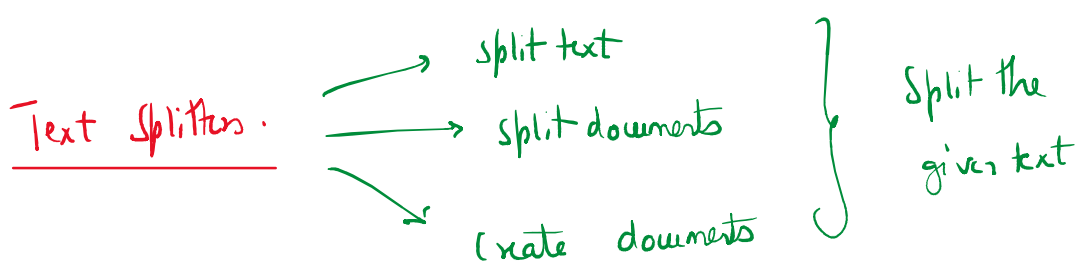


loss of informations.

③ embedding models $\rightarrow$ 1 vector.

20 parts $\rightarrow$ parallelly

Batches



I/P $\rightarrow$ O/P

split text I/P str O/P list[str]

split documents list[Doc] list[Documents]

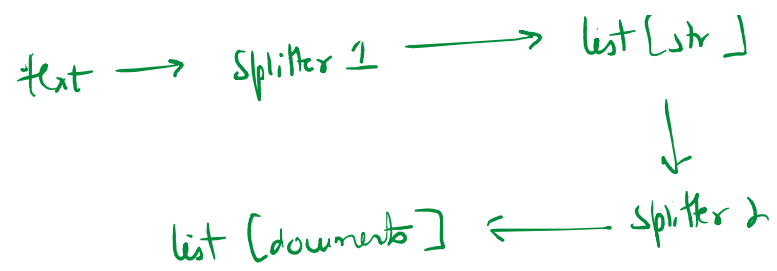
create documents list(str) list[documents]

1000 words $\downarrow$ 10 chunks

100 pages $\downarrow$ 100 docs $\downarrow$ 10 parts

500 doc

text $\rightarrow$ splitter 1 $\rightarrow$ list[str]



Length Based Splitting

What is it?

Fixed-size chunking is the simplest strategy where you split text into chunks of a predetermined size (either by character count or token count). You can optionally include an overlap between chunks to preserve context at chunk boundaries.

ln

Theory

Code

Advantages

Disadvantages

Text → length
 ↘
 character count
 ↘
 Tokens

length. → 100 char

length of chunk

Threshold → chunk size maximum

separator → character , words , lines , paragraph
 " " " " " \n " " \n \n "

200

Advantages:

- Simple and straightforward to implement
- Predictable chunk sizes fit model context windows
- Fast processing speed
- Consistent memory usage
- Easy to parallelize

chunk size, overlap,
sep

Text → 100
 C1 | C2 | C3
 100 | 100 | 100

Disadvantages:

- May split sentences or paragraphs mid-thought, breaking semantic meaning
- No consideration of document structure or meaning
- Can separate related information across chunks
- Overlap creates data redundancy
- Not optimal for documents with varied structure

words break

context break

3 paragraphs
 ↓
 sentences

Book → chapters

9

Chunk X
 Physics
 Chem

Context Chunks → 2 para

X

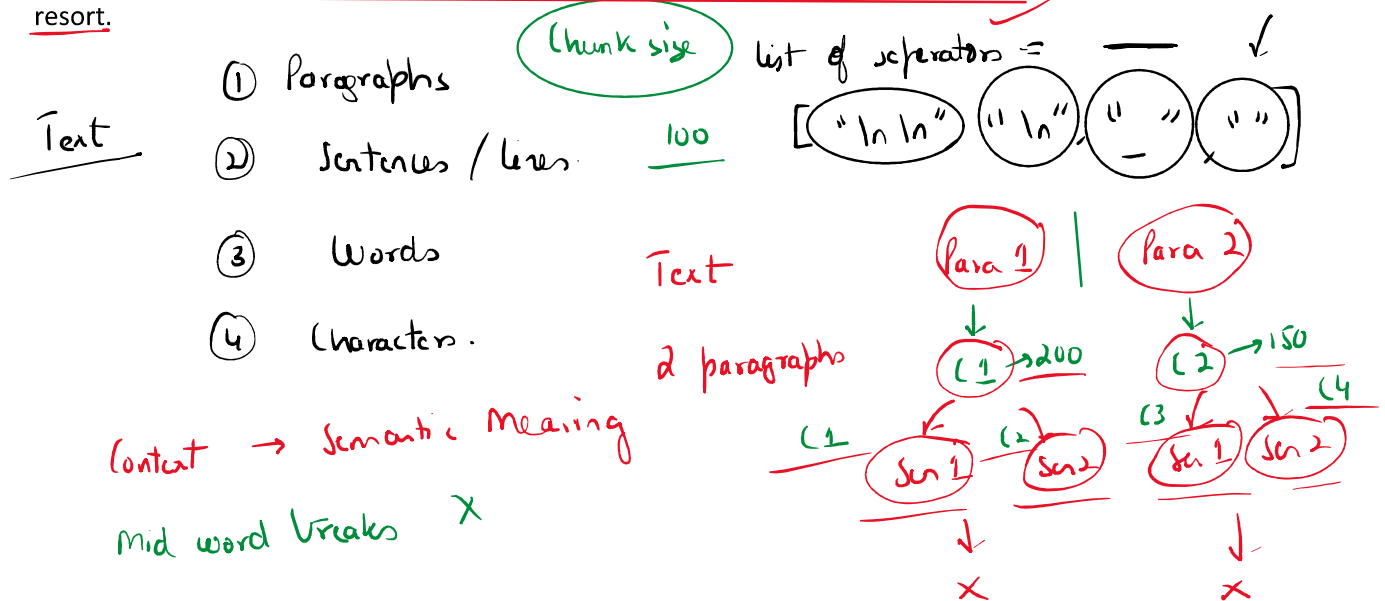
Structure

Context continuity

① Mid sentence, Mid word② Document Structure**What is it?**

Recursive chunking is the **most recommended strategy** in LangChain. It attempts to split text hierarchically using a list of separators, starting with larger units (paragraphs) and recursively moving to smaller units (sentences, then words) if chunks are still too large.

Think of it as a smart scissors that tries to cut at natural boundaries - first trying paragraph breaks, then sentence boundaries, and only cutting mid-sentence as a last resort.

**Advantages:**

- Best balance between chunk size and semantic coherence
- Preserves natural language structure
- Respects document hierarchy (paragraphs → sentences → words)
- Recommended for most use cases
- Configurable separators for different document types
- Maintains context better than fixed-size chunking

Disadvantages:

- More complex than simple fixed-size splitting
- May still break semantic units if they're too long
- Requires careful separator selection
- Can create variable chunk sizes
- May not work well with highly structured documents

Recursive chunker

Markdown

#

=

-

class

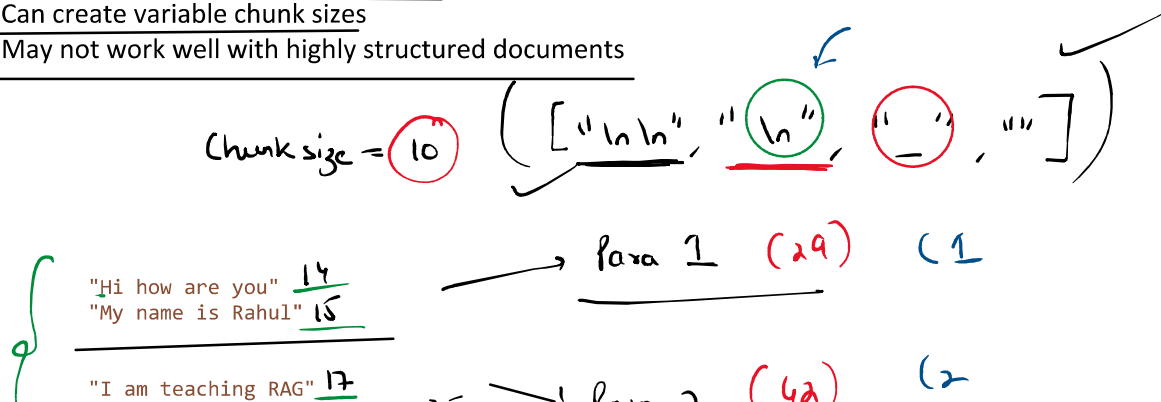
def

for

if

**

**



My name is Rahul 15
"I am teaching RAG" 17
"We are Learning about RAG" 25

Para 2 (42) (2)

(1)
Hi how are you
My name is Rahul

(2)
I am teaching RAG
We are learning about RAG

(1)
Hi how are you
(14) ✓

(2)
My name is Rahul
(15) ✓

(3)
I am teaching RAG
(17) ✓

(4)
We are
learning about
RAG
(25) ✓

Hi how are you
(2) (3) (3) (3)

My name is Rahul
(2) (4) (2) (5) X

Hi how are
(2) (10)

[you] My Name is Rahul
(2) (3) (10) (8) (4)

merge text

python, Markdown,
JSON text

list of sep

What is it?

This strategy leverages the inherent structure of documents (HTML tags, Markdown headers, JSON objects, code functions) to create meaningful chunks. Each chunk respects the logical organization of the document.

It's like organizing a book by chapters and sections rather than just cutting it every 1000 words.

< head > ——— < / head >
< div > ——— < / div >

h1 ✓
h2 ✓
h3 , h6

Structured

{ "name": "Amit",

"contact": { "email": ——— ,
"mobile": ——— , }

"fav-color": [———] }

def my-func():

return ———

class, def, if, for

class MyClass:

—————
—————
—————

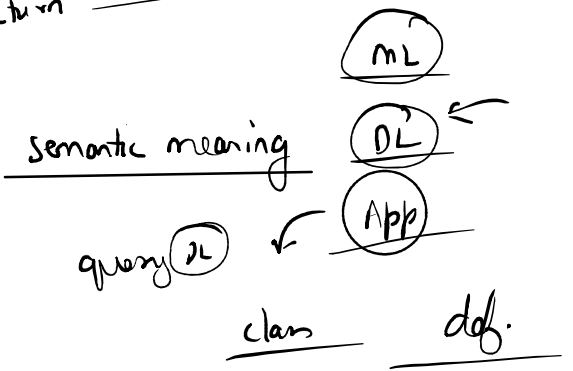
—————
—————
—————

Advantages:

- Preserves logical document organization
- Maintains semantic meaning within chunks
- Metadata-rich chunks (headers, section info)
- Excellent for structured documents
- Better retrieval accuracy
- Natural chunk boundaries

Disadvantages:

- Only works with structured documents
- Requires document-specific splitters
- May create very large chunks if sections are long
- More complex to implement
- Not suitable for unstructured text
- May need combination with other strategies



What is it?

Semantic chunking creates chunks based on the meaning and semantic similarity of the text rather than arbitrary size limits. It starts with an initial chunk and keeps adding sentences as long as they remain semantically similar (measured by cosine similarity of embeddings). When similarity drops below a threshold, a new chunk begins.

This is like grouping related topics in a conversation - you keep talking about one subject until the topic naturally shifts.

Chunk size λ

$t \rightarrow$ chunk chunks
 $\swarrow \downarrow \searrow$
 similarity
 semantic

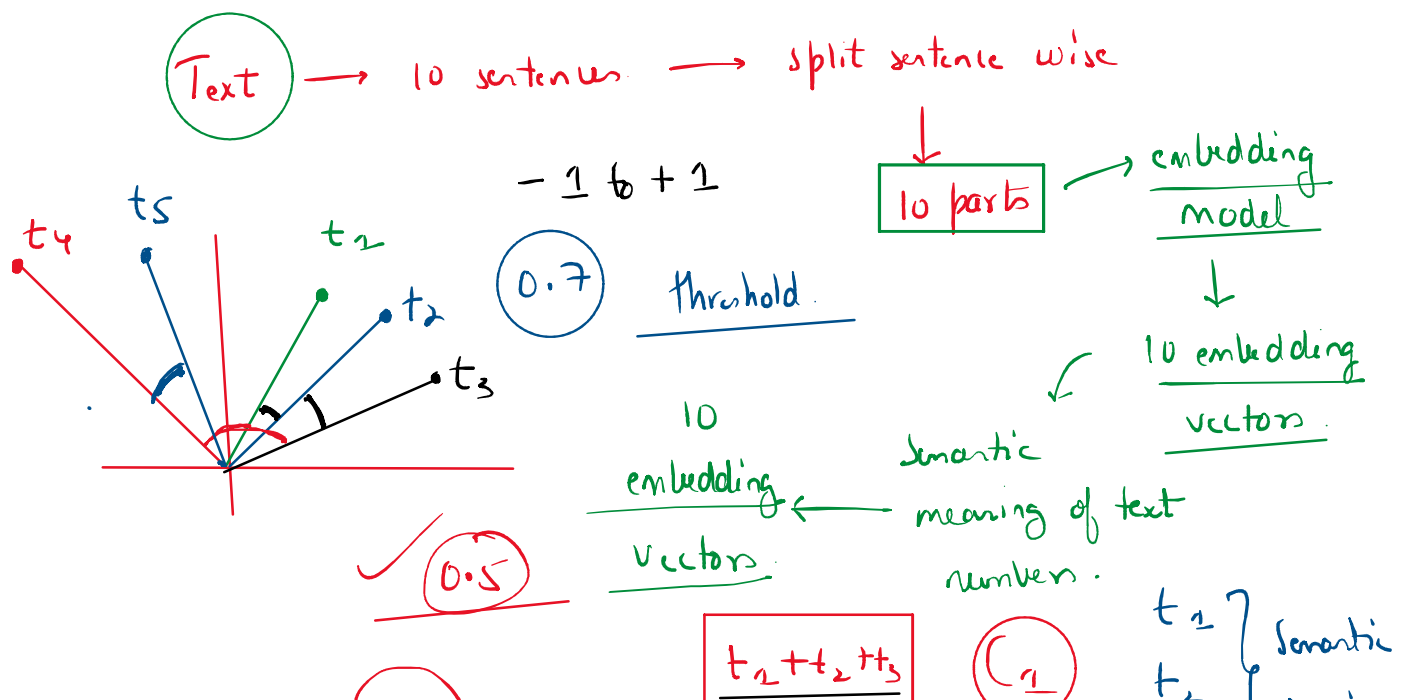
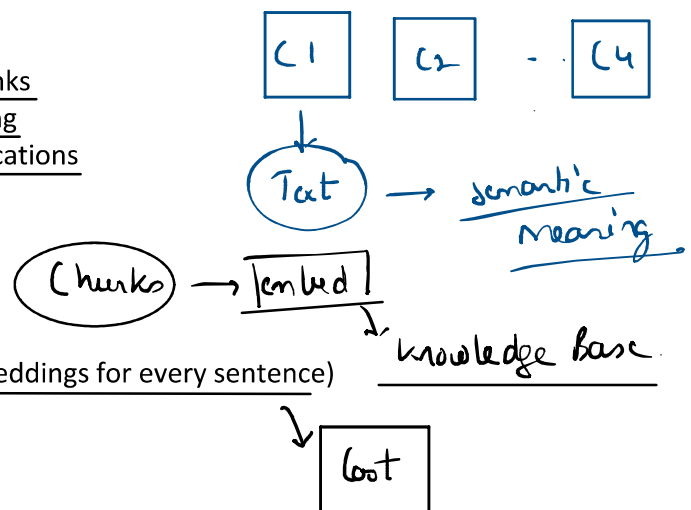
Meaning of txt

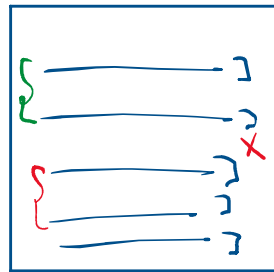
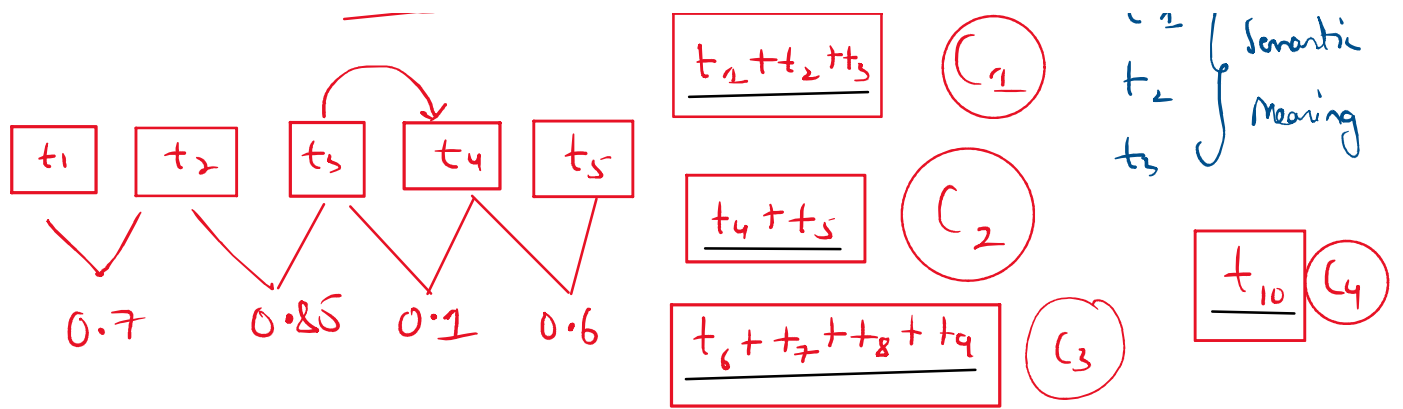
Advantages:

- Preserves semantic coherence within chunks
- More intelligent splitting based on meaning
- Better retrieval performance in RAG applications
- Natural topic boundaries
- Improved context preservation

Disadvantages:

- Computationally expensive (requires embeddings for every sentence)
- Variable chunk sizes can be problematic
- Slower processing time
- Requires an embedding model
- May create very large or very small chunks
- Difficult to predict chunk count in advance

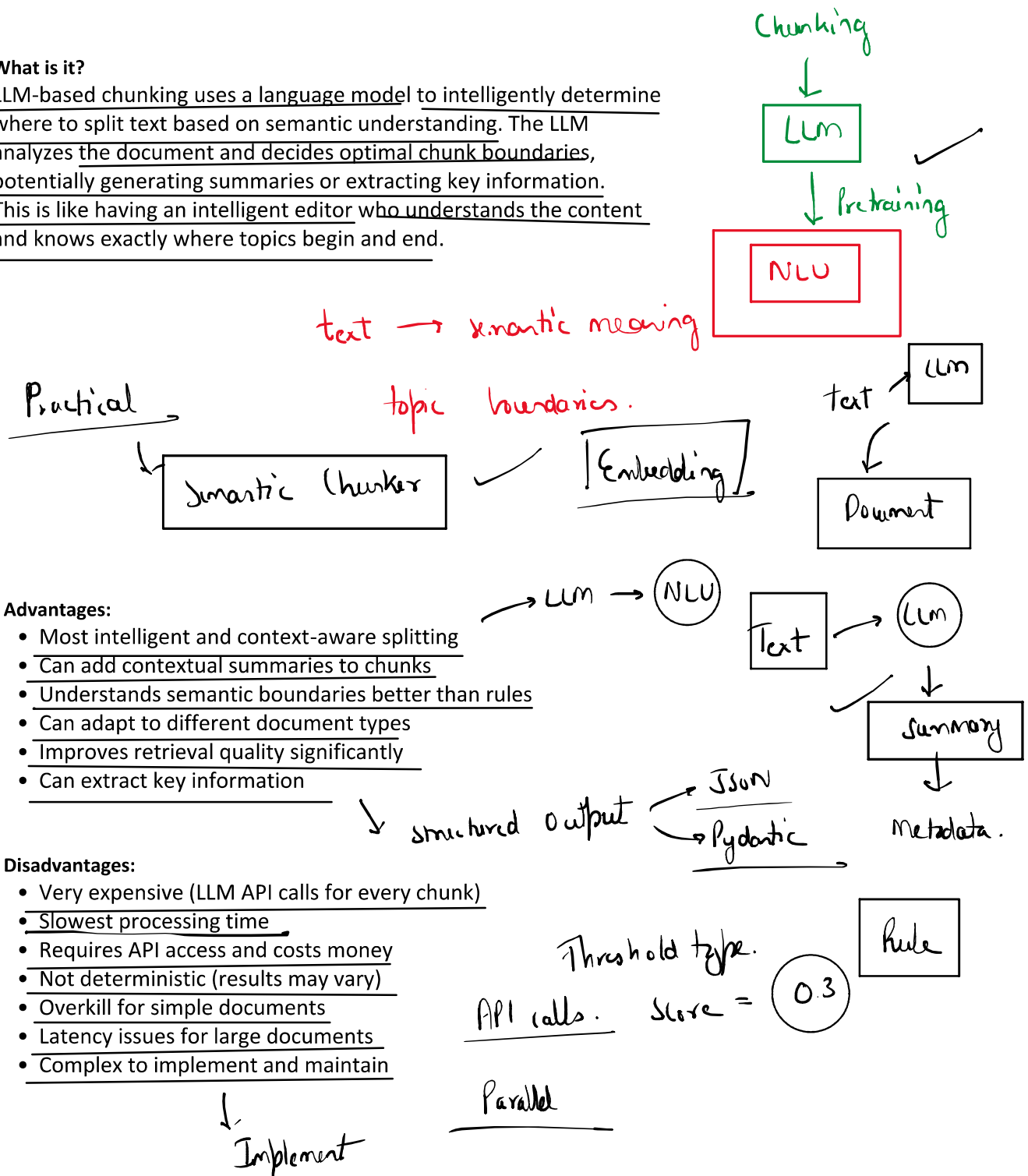


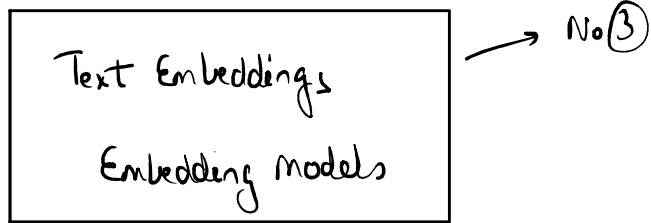


semantic similarity

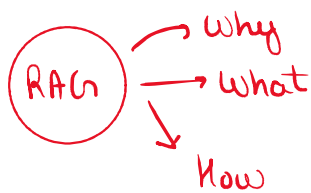
What is it?

LLM-based chunking uses a language model to intelligently determine where to split text based on semantic understanding. The LLM analyzes the document and decides optimal chunk boundaries, potentially generating summaries or extracting key information. This is like having an intelligent editor who understands the content and knows exactly where topics begin and end.





Knowledge source.



External knowledge

↓
Data ingestion

Document loaders.



Unified format

Document

Page content Metadata.

Text splitter. → Documents
↓
Chunks

5 Chunking Strategies

① Character length Based Splitting → Text → chunk size
↓
chunk
Contextual continuity
↓
Character count

② Text Structure Based Splitting → Text
↓
Paragraphs Lines Words Characters

③ Document structure based splitting → Md, HTML, JSON, Code files.
Separators: Python. Class.

Separators:

Headers:

Python.

Class.

ib dse

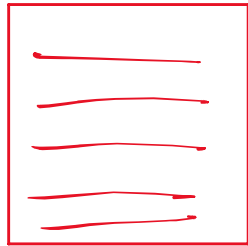
def

④ Semantic chunking

Semantic meaning of text

Threshold :

0.5



0.7

0.2

C_1 C_2



↓
Similar contextual meaning.

embedding model

vector.

(semantic meaning)

→ similarity

↓
numbers.

Chunking

⑤ Lm Based chunking

Text

LLM → structured output

NLU

Semantic meaning change.

Pydantic model
(schema)

$[\text{Document}] \rightarrow [\text{chunk 1}, \text{chunk 2} \dots]$

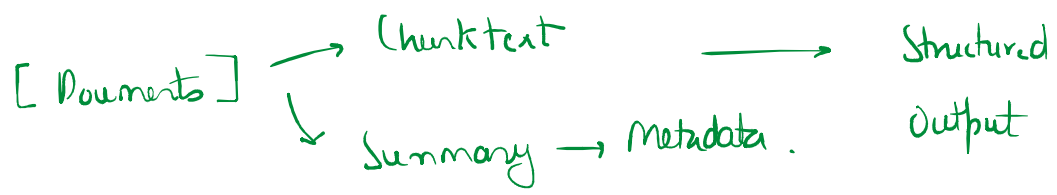
Document

Page $\rightarrow$ Text
context

Metadata. → summary of chunk.

[Documents] $\rightarrow$ chunktext

Structured



Categorical Col $\xrightarrow{\text{encode}}$ Numbers $\begin{cases} \rightarrow \text{One} \\ \rightarrow \text{Ordinal} \end{cases}$

Text X $\xrightarrow{\text{Similarities}}$ Numbers.

$\rightarrow \text{ML} \times$

Text $\rightarrow$ Vectors

Stage 1:- Classical Methods (Bow, TF-IDF) $\rightarrow$ Sparse Vectors.
 Count the frequency of words in your text

English lang $\rightarrow$ (10k) Vocabulary $\rightarrow$ Unique set of words.

	Cat	Sat	Mat	Rat	Wall	Ball
Text 1	<u>1</u>	<u>1</u>	<u>1</u>	0	0	0
(Cat) (Sat) on the (mat)	(2)	0	(3)	(2)	0	(1)
Text 2						

5✓

Sparse vector $\rightarrow$ Most of the values are zero.

1 x 10,000

$\rightarrow$ Non zero.

(10k)

Similarity $\rightarrow$ calculations (Element wise)

Stage 2 → Word Embeddings (Dense vectors)



Information

Number of calculations ↓

Dense Vectors.

① Reduced Dimensionality

② Each features holds info.

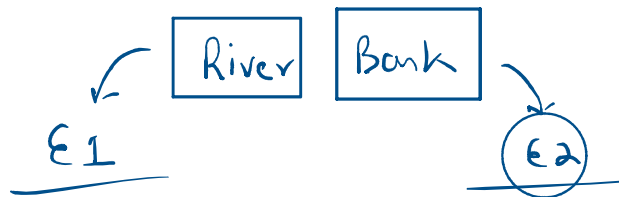
Semantic Meaning

768

Word by word

→ Location

768



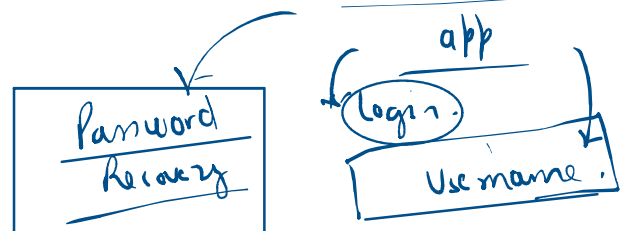
Contextual Meaning

→ institution

E3

E2

Query → chunk X



Stage 3 → Contextual Embeddings (Dense, contextual meaning)

Context - aware embeddings.

BERT,

Open AI,

Gemma Embeddings,

Sentence Transformers,

Money Bank

→ E_1

River Bank

✓
→

Sentences

Paragraphs

↓

numbers.

The Big Picture

